

CPU 并发加速方案 —— 提取页面阶段

背景：GPU 占用率低已确认为任务管理器显示问题（ncnn-vulkan 走 Compute 队列），无需再修。
现发现 [1/3] 提取页面 阶段 CPU 占用率低、速度偏慢，本文基于**本机实测数据**给出并发加速方案与优劣分析。

实测环境：Windows，16 核 CPU，PyMuPDF 1.28.2，zoom 2.0，测试 PDF 173 页（张宇 30 讲概率）。

一、现状与瓶颈定位

当前 scripts/extract_pages.py 是单进程单线程逐页处理：

```
for 每页: load_page(i) → get_pixmap(zoom 2.0) → pix.save(page_NNNN.png)
```

16 核机器只有 1 个核在干活，这就是"CPU 占用率低"的直接原因。

实测耗时分解（本机）：

环节	单页耗时	占比
渲染 get_pixmap(zoom 2.0)	~100 ms	~27%
PNG 编码 + 写盘 pix.save	~264 ms	~73%
顺序合计	~367 ms	100%

结论：瓶颈在 PNG 编码/写盘，其次是渲染；两者都是 C 层密集计算，正是并发可以吃掉的部分。

并行实测对比（64 页样本，同机）：

方式	总耗时	每页	加速比	说明
顺序（现状）	23.48 s	367 ms	1.0x	—
多线程 x4	3.07 s（8页）	~384 ms	~1.0x	几乎无收益，且曾触发进程异常终止 (0xC000013A)
多进程 x4	7.17 s	112 ms	3.3x	接近线性
多进程 x8	4.42 s	69 ms	5.3x	16 核仍有富余

二、方案对比

方案 A：多进程并行（ProcessPoolExecutor）—— 推荐

原理：每个 worker 是独立进程，各自 `fitz.open(pdf)` 后负责一批页面，渲染+保存全程并行；多进程天然绕开 GIL，且 PyMuPDF 原生调用在各进程内独立、无跨线程共享的线程安全问题。

实现要点：

- 按页号分块分配（如 worker k 处理 `k, k+w, k+2w, ...`），保证输出文件名 `page_%04d.png` 与页号一一对应，**并行不影响成品页序**；
- Windows 上 `ProcessPoolExecutor` 用 `spawn` 启动，worker 函数必须放在模块顶层（可 pickle），主流程放在 `if __name__ == "__main__":` 下；
- 内存：每进程一个文档 + 当前页 pixmap（zoom 2.0 约 6 MB/页），8 worker 峰值远小于 1 GB，可控；
- 页数很少（<16）时进程启动开销（每 worker 需 import fitz，约 0.2~0.3 s）占比高，收益小；页数多时接近线性。

优：加速明显（实测 3.3~5.3 倍）；进程隔离、最安全；改动集中在 `extract_pages.py` 一个脚本（新增 `--workers N`），`run.py` 无需改动。

劣：进程启动/导入有固定开销；内存按 worker 数倍增；磁盘写并发在机械盘上可能轻微抖动（SSD 无感）。

建议：`workers` 默认 `min(8, os.cpu_count())`；16 核机器取 8 性价比最好。

方案 B：多线程并行（ThreadPoolExecutor）—— 不推荐

原理：靠 PyMuPDF 的 C 层渲染/编码释放 GIL 来实现"伪并行"。

实测结果：4 线程 ≈ 顺序，**零收益**。原因是 PyMuPDF 的原生调用存在内部锁/GIL 串行化，并发请求被排队；同时原生层线程安全并不保证，组合测试中曾出现进程被终止（0xC000013A）。

结论：排除，不做多线程方案。

方案 C：降低 PNG 编码开销（可与方案 A 叠加）—— 可选优化

实测 PNG 编码速度（单页，zoom 2.0）：

保存方式	单页耗时	文件大小
mupdf <code>pix.save()</code> （现状）	~264 ms	2.0 MB
PIL <code>compress_level=1</code>	~236 ms	1.8 MB
PIL <code>compress_level=6</code>	~646 ms	1.6 MB
PIL <code>compress_level=9</code>	~1.6 s	1.6 MB

- 现状 mupdf 默认已接近 `level=1` 的速度，单纯换成 PIL level 1 只省 ~10%，意义不大；
- 想再快只能上 **JPEG**（质量 90，编码 ~50 ms），但会引入有损，超分引擎会放大压缩伪影，**扫描件一般不建议**；若坚持，注意 `rebuild_pdf.py` 与超分引擎均支持 jpg 输入，改起来不难；
- 结论：作为"锦上添花"，与方案 A 叠加后，提取阶段总耗时的瓶颈会转向磁盘 IO，继续压编码收益递减。

方案 D：磁盘 IO 侧 —— 通常无需处理

并行写同一目录在 SSD 上无感；若输出到机械盘/网络盘，可考虑按 worker 分目录写、完成后合并（一般不必）。

三、推荐落地方式（最小改动）

只改 `scripts/extract_pages.py`：

1. 新增 `--workers N` 参数（默认 1，完全保持现状行为；`N>1` 时走 `ProcessPoolExecutor`）；
2. 主进程打开 PDF 拿 `page_count`，把页号按 stride 分块分给各 worker；
3. 每个 worker 自己 `fitz.open(pdf)`（各自进程内实例，安全），渲染+`pix.save`；
4. 主进程回收异常：任一 worker 失败则整体报错退出，避免缺页成品；
5. `run.py` 不变：仍以子进程调用 `extract`，`count_png(src)` 轮询进度、实时状态栏照常工作（页数不变，只是更快）。

预期（本机 173 页）：顺序约 **64 s** → 8 worker 约 **12 s**（5 倍加速）。

四、风险与注意事项

1. **不要用多线程方案**：无收益且有原生崩溃风险（实测 `0xC000013A`）。
2. worker 数不宜远超物理核数（16 核建议 $\leq 8\sim 12$ ），多了内存/调度开销反噬。
3. 与下一阶段（GPU 超分）错峰使用资源：提取结束、超分开始后 GPU 独享，互不干扰。
4. `fitz.TOOLS` 之类的全局设置在各 worker 进程内各自生效，无需跨进程同步。
5. 测试中出现的 `MuPDF error: cannot create appearance stream for annotations` 是该 PDF 自带的批注问题，**与并行无关**，可忽略。
6. Anaconda/杀软环境下多进程首次启动稍慢，属正常现象。